

Συστήματα Μικροϋπολογιστών

3Η ΟΜΑΔΑ ΑΣΚΗΣΕΩΝ

Διονύσης Κατσέτης el23005

Ασκήσεις προσομοίωσης της αρχιτεκτονικής ARM

ΑΣΚΗΣΗ 1

Υλοποίηση με c

init.s: Κώδικας βοηθητικός για την main.c

```
.syntax unified
.thumb
.text

.global Hardware_Init
.global UART2_SendChar
.global UART2_ReceiveChar
.global Check_Button_PD4

.equ RCC_BASE,      0x40023800
.equ RCC_AHB1ENR,   0x30
.equ RCC_APB1ENR,   0x40

.equ GPIOA_BASE,     0x40020000
.equ GPIOD_BASE,     0x40020C00
.equ GPIO_MODER,     0x00
.equ GPIO_PUPDR,     0x0C
.equ GPIO_IDR,       0x10
```

```
.equ GPIO_AFRL, 0x20
```

```
.equ USART2_BASE, 0x40004400
```

```
.equ USART_SR, 0x00
```

```
.equ USART_DR, 0x04
```

```
.equ USART_BRR, 0x08
```

```
.equ USART_CR1, 0x0C
```

```
.type Hardware_Init, %function
```

```
.type UART2_SendChar, %function
```

```
.type UART2_ReceiveChar, %function
```

```
.type Check_Button_PD4, %function
```

@ Συμπληρωματική: Hardware_Init

Hardware_Init:

```
push {lr}
```

```
ldr r0, =RCC_BASE
```

@ Clock Activation for GPIOA (Bit 0) and GPIOD (Bit 3) in AHB1

```
ldr r1, [r0, #RCC_AHB1ENR]
```

```
orr r1, r1, #0x09 @ 0x09 = 1001 binary
```

```
str r1, [r0, #RCC_AHB1ENR]
```

@ Clock Activation for USART2 (Bit 17) in APB1

```
ldr r1, [r0, #RCC_APB1ENR]
```

```
orr r1, r1, #(1 << 17)
```

```
str r1, [r0, #RCC_APB1ENR]
```

@ GPIOA

```
ldr r0, =GPIOA_BASE
```

```
ldr r1, [r0, #GPIO_MODER]
```

@ PA12 & PA13 -> Digital Outputs (Mode 01)

```
bic r1, r1, #(0xF << 24) @ clear 24-27 bits first
```

```
orr r1, r1, #(0x5 << 24)
```

@ PA2 (TX) & PA3 (RX) -> Alternate Function (Mode 10)

```
bic r1, r1, #(0xF << 4) @ first clear 4-7 bits
```

```
orr r1, r1, #(0xA << 4)
```

```
str r1, [r0, #GPIO_MODER]
```

@ Connect PA2, PA3 with Alternate Function 7 (USART2)

```
ldr r1, [r0, #GPIO_AFRL]
```

```
bic r1, r1, #(0xFF << 8) @ clear first 8-15 bits
```

```
orr r1, r1, #(0x77 << 8)
```

```
str r1, [r0, #GPIO_AFRL]
```

@ GPIOD

```
ldr r0, =GPIOD_BASE
```

```
ldr r1, [r0, #GPIO_MODER]
```

@ PD4 Digital Input (Mode 00).

```
bic r1, r1, #(0x3 << 8)
str r1, [r0, #GPIO_MODER]
```

@ Pull-Down for PD4

```
ldr r1, [r0, #GPIO_PUPDR]
bic r1, r1, #(0x3 << 8)
orr r1, r1, #(0x2 << 8) @ 10 = Pull-Down
str r1, [r0, #GPIO_PUPDR]
```

@ USART2

```
ldr r0, =USART2_BASE
```

@ Deactivation of USART2 before configuration

```
mov r1, #0
str r1, [r0, #USART_CR1]
```

@ Baud Rate (Γα 9600 bps for default clock 16MHz)

```
@ 16.000.000 / (16 * 9600) = 104.166 -> 104 (0x68) Mantissa, 3 (0x3) Fraction = 0x0683
ldr r1, =0x0683
str r1, [r0, #USART_BRR]
```

@ Activation of USART (UE=1), Transmitter (TE=1), Receiver (RE=1)

@ Bits: UE=13, TE=3, RE=2 -> 0x200C

```
ldr r1, =0x200C
str r1, [r0, #USART_CR1]
```

```
pop {pc}
```

@ UART2_SendChar

UART2_SendChar:

```
ldr r1, =USART2_BASE
```

Wait_TX:

```
ldr r2, [r1, #USART_SR]
tst r2, #(1 << 7)
beq Wait_TX
```

```
str r0, [r1, #USART_DR]
```

```
bx lr
```

@ UART2_ReceiveChar

UART2_ReceiveChar:

```
ldr r1, =USART2_BASE
```

```
ldr r2, [r1, #USART_SR]
```

```
tst r2, #(1 << 5)
```

```
beq No_Data
```

```
ldr r0, [r1, #USART_DR]
```

```
bx lr
```

No_Data:

```
mov r0, #0
bx lr
```

@ Check_Button_PD4

Check_Button_PD4:

```
ldr r1, =GPIO_BASE
ldr r2, [r1, #GPIO_IDR]
tst r2, #(1 << 4)
bne Button_Pressed
```

```
mov r0, #0
bx lr
```

Button_Pressed:

```
mov r0, #1
bx lr
```

Ανάλυση *init.s*:

1. Σκοπός και Αρχιτεκτονική Περιγραφή

Ο κώδικας του *init.s* υλοποιεί την επικοινωνία με το υλικό, γραμμένο σε **ARM Assembly (Thumb-2)** για τον επεξεργαστή Cortex-M4. Η λειτουργικότητά του εστιάζει στον προγραμματισμό των περιφερειακών μονάδων, τη διαχείριση της σειριακής θύρας (UART2) και τον έλεγχο των σημάτων εισόδου από το χρήστη.

2. Ανάλυση Περιφερειακών και Διαμόρφωση Καταχωρητών

A. Συγχρονισμός και Διανομή Ρολογιού (RCC Configuration)

Χρησιμοποιώντας τον καταχωρητή `RCC_AHB1ENR`, παρέχεται τροφοδοσία ρολογιού στις θύρες **GPIOA** και **GPIOB**. Επιπλέον, μέσω του `RCC_APB1ENR`, ενεργοποιείται η λειτουργία του **USART2** στον αντίστοιχο δίαυλο χαμηλής ταχύτητας.

B. Παραμετροποίηση Θυρών Εισόδου/Εξόδου (GPIO Configuration)

- **Θύρα A (GPIOA):** Οι ακροδέκτες **PA12** και **PA13** ορίζονται ως εξόδοι για τον έλεγχο των LED. Τα **PA2 (TX)** και **PA3 (RX)** συνδέονται με τον USART2 μέσω της εναλλακτικής λειτουργίας **AF7**.
- **Θύρα D (GPIOB):** Ο ακροδέκτης **PD4** ρυθμίζεται ως ψηφιακή είσοδος με ενεργοποιημένη την αντίσταση **Pull-Down** για την ανάγνωση του διακόπτη.

Γ. Ρύθμιση Σειριακής Διεπαφής (USART2 Configuration)

Η μονάδα USART2 διαμορφώνεται για επικοινωνία στα **9600 bps**, με τον καταχωρητή `USART_BRR` να δέχεται την τιμή `0x0683`. Η πλήρης ενεργοποίηση του περιφερειακού επιτυγχάνεται μέσω του `USART_CR1`.

3. Περιγραφή Συναρτήσεων (API)

- `Hardware_Init`: Συνολική αρχικοποίηση του συστήματος (ρολόγια, GPIO, UART).
- `UART2_SendChar`: Αποστολή byte μέσω ελέγχου του bit **TXE**.
- `UART2_ReceiveChar`: Μη-μπλοκαριστική λήψη δεδομένων με έλεγχο του **RXNE**.
- `Check_Button_PD4`: Άμεση ανάγνωση της κατάστασης της εισόδου PD4.

(Δεν χρησιμοποιήθηκε από την `main.c` αφού σε `c` θέλουμε να μη κάνει `polling` αλλά το πάτημα του να προκαλεί `hardware interrupt`.)

main.c:

```
#include <stdint.h>
#include <stdio.h>
#include <string.h>
// External Assembly Functions (from init.s)
extern void Hardware_Init(void);
extern void UART2_SendChar(char c);
extern char UART2_ReceiveChar(void);
#define GPIOA_ODR    (*(volatile uint32_t *)0x40020014)
#define LED_PA12     (1 << 12)
#define LED_PA13     (1 << 13)
#define RCC_APB2ENR  (*(volatile uint32_t *)0x40023844)
#define SYSCFG_EXTICR2 (*(volatile uint32_t *)0x4001380C)
#define EXTI_IMR     (*(volatile uint32_t *)0x40013C00)
#define EXTI_RTISR   (*(volatile uint32_t *)0x40013C08)
#define EXTI_PR      (*(volatile uint32_t *)0x40013C14)
#define NVIC_ISER0   (*(volatile uint32_t *)0xE000E100)
#define MAX_INPUT_LEN 256
static const char covid_seq[] =
"AUUAAAGGUUUAUACCUUCCAGGUAACAAACCAACCAACUUUCGAUCUCUUGUAGAUCUGUUCUCUAAACGAACUUUA ..."
(Παραλείπουμε το υπόλοιπο καθώς είναι τεράστιο)
char input_buf[MAX_INPUT_LEN];
uint32_t input_idx = 0;
volatile int reset_search_flag = 0;
// Helper Functions
void UART2_SendString(const char* str) {
    while (*str) {
        UART2_SendChar(*str++);
    }
}
char get_complement(char c) {
    switch(c) {
        case 'A': return 'U';
        case 'U': return 'A';
        case 'G': return 'C';
        case 'C': return 'G';
        default: return 0;
    }
}
void delay_ms(uint32_t ms) {
```

```

    for (volatile uint32_t i = 0; i < (ms * 1600); i++) {
        __asm__("nop");
    }
}

void Interrupt_Init(void) {
    // Enable SYSCFG clock (Bit 14)
    RCC_APB2ENR |= (1 << 14);
    // Route PD4 to EXTI4 in SYSCFG_EXTICR2 (Bits [3:0] = 0011 for Port D)
    SYSCFG_EXTICR2 &= ~(0x000F); // Clear bits
    SYSCFG_EXTICR2 |= 0x0003; // Set to 0x3 (Port D)
    // 3. Unmask EXTI4 line
    EXTI_IMR |= (1 << 4);
    // 4. Enable Rising Edge Trigger for EXTI4
    EXTI_RTSR |= (1 << 4);
    // 5. Enable EXTI4_IRQn in NVIC (Position 10 in ISER0)
    NVIC_ISER0 |= (1 << 10);
}

// The Hardware ISR for EXTI Line 4
void EXTI4_IRQHandler(void) {
    // Verify the interrupt was triggered by EXTI4
    if (EXTI_PR & (1 << 4)) {
        reset_search_flag = 1;
        // Clear the pending bit by writing '1'
        EXTI_PR = (1 << 4);
    }
}

int main(void) {
    // Initialize hardware via Assembly routine
    Hardware_Init();
    // Initialize Pure Hardware Interrupt for PD4 -> EXTI4
    Interrupt_Init();
    UART2_SendString("SARS-CoV-2 RNA Matcher Initialized.\r\n");
    UART2_SendString("Enter RNA sequence (A, G, U, C):\r\n");
    while (1) {
        // Interrupt Handler Check
        if (reset_search_flag) {
            GPIOA_ODR |= LED_PA13; // Turn ON PA13
            delay_ms(3000); // Delay (Safe in Thread Mode)
            GPIOA_ODR &= ~LED_PA13; // Turn OFF PA13
            input_idx = 0;
            memset(input_buf, 0, sizeof(input_buf));
            UART2_SendString("\r\n[RESET] Operation aborted. Enter new sequence:\r\n");
            reset_search_flag = 0; // Acknowledge flag
        }
        // UART Polling (Non-blocking)
        char rx = UART2_ReceiveChar();
        if (rx != 0) {
            if (rx == 'A' || rx == 'G' || rx == 'U' || rx == 'C') {
                if (input_idx < MAX_INPUT_LEN - 1) {
                    input_buf[input_idx++] = get_complement(rx);
                    UART2_SendChar(rx); // Echo for user interface
                }
            }
        }
        else if (rx == '\n' || rx == '\r') {
            if (input_idx > 0) {
                input_buf[input_idx] = '\0';
                UART2_SendString("\r\nSearching for complement: ");
                UART2_SendString(input_buf);
                UART2_SendString("\r\n");
                int matches = 0;
                char *ptr = covid_seq;
            }
        }
    }
}

```

```

char tx_buf[64];
while ((ptr = strstr(ptr, input_buf)) != NULL) {
    matches++;
    int position = (ptr - covid_seq) + 1;
    printf(tx_buf, "Match found at position: %d\r\n", position);
    UART2_SendString(tx_buf);
    ptr++;
}
printf(tx_buf, "Total matches: %d\r\n\r\n", matches);
UART2_SendString(tx_buf);
if (matches > 0) {
    GPIOA_ODR |= LED_PA12;
    delay_ms(2000);
    GPIOA_ODR &= ~LED_PA12;
}
input_idx = 0;
memset(input_buf, 0, sizeof(input_buf));
UART2_SendString("Enter RNA sequence:\r\n");
}
}
}
}
}

```

Ανάλυση main.c:

1. Σκοπός και Αρχιτεκτονική Περιγραφή

Το αρχείο main.c αποτελεί τον κεντρικό πυλώνα του συστήματος **SARS-CoV-2 RNA Matcher**. Η εφαρμογή επιτυγχάνει τη λήψη αλληλουχιών μέσω UART2, τον υπολογισμό του συμπληρωματικού σκέλους και την εκτέλεση αναζήτησης εντός του ιογενούς γονιδιώματος. Ο σχεδιασμός συνδυάζει έναν non-blocking βρόχο ελέγχου για τη σειριακή επικοινωνία με τη χρήση εξωτερικών διακοπών (EXTI) για τον άμεσο έλεγχο της κατάστασης του συστήματος.

2. Ανάλυση Μηχανισμού Διακοπών (Hardware EXTI4 & NVIC)

Για την υλοποίηση της λειτουργίας Reset μέσω του διακόπτη PD4, χρησιμοποιείται η γραμμή EXTI4 (καθώς είναι αδύνατο να γίνει με τη *EXTI0 hardware interrupt πατώντας το PD4*) με την εξής παραμετροποίηση:

- **RCC & SYSCFG:** Ενεργοποιείται η τροφοδοσία του SYSCFG και ρυθμίζεται ο καταχωρητής SYSCFG_EXTICR2 για τη σύνδεση της θύρας D με τη γραμμή διακοπής 4.
- **EXTI Configuration:** Γίνεται απομάσκωση μέσω του EXTI_IMR και ορίζεται η πυροδότηση σε **Rising Edge** στον EXTI_RTSR.
- **NVIC Activation:** Η διακοπή ενεργοποιείται στον ελεγκτή NVIC μέσω του καταχωρητή NVIC_ISER0 (Bit 10).
- **ISR (EXTI4_IRQHandler):** Ο Handler θέτει τη σημαία reset_search_flag και καθαρίζει το αίτημα διακοπής στον EXTI_PR.

3. Διαχείριση Μνήμης και Αλγόριθμος Αναζήτησης

A. Βελτιστοποίηση Χώρου Μνήμης (Flash vs RAM)

Για την εξοικονόμηση πόρων, η ακολουθία RNA αποθηκεύεται ως **static const**. Αυτή η επιλογή εξασφαλίζει την παραμονή των δεδομένων στη μνήμη **Flash**, αποφεύγοντας την υπερφόρτωση της RAM.

B. Επεξεργασία και Σειριακή Ροή

Η λειτουργία του συστήματος στον κύριο βρόχο περιλαμβάνει τα εξής στάδια:

- **Φιλτράρισμα & Συμπλήρωση:** Οι εισερχόμενοι χαρακτήρες μετατρέπονται μέσω της `get_complement()` και αποθηκεύονται στον εσωτερικό buffer με ταυτόχρονη εμφάνιση (`echo`).
- **Αλγόριθμος strstr:** Μετά τον τερματισμό της εισαγωγής, η αναζήτηση πραγματοποιείται με τη συνάρτηση `strstr()`, ενώ η θέση κάθε ταύτισης υπολογίζεται με αριθμητική δεικτών.

3. Hardware Feedback: Η εύρεση αποτελεσμάτων σηματοδοτείται από το LED στον **PA12**.

Αντίστοιχα, σε περίπτωση ακύρωσης μέσω διακοπής, εκκαθαρίζεται ο buffer και ενεργοποιείται το LED στον **PA13** για οπτική επιβεβαίωση της επανεκκίνησης.

Υλοποίηση με assembly

```
.syntax unified
.thumb
.text

.global main

from init.s
.extern Hardware_Init
.extern UART2_SendChar
.extern UART2_ReceiveChar
.extern Check_Button_PD4

.equ GPIOA_ODR,    0x40020014
.equ LED_PA12,     (1 << 12)
.equ LED_PA13,     (1 << 13)
.equ MAX_INP_LEN,  256

.section .rodata
msg_init: .asciz "SARS-CoV-2 RNA Matcher Initialized (Polling Mode).\r\n"
msg_prompt: .asciz "Enter RNA sequence (A, G, U, C):\r\n"
msg_search: .asciz "\r\nSearching for complement: "
```



```

msg_abort: .asciz "\r\n[RESET] Operation aborted. Enter new sequence:\r\n"
msg_match: .asciz "Match found at position: "
msg_total: .asciz "Total matches: "
msg_crlf: .asciz "\r\n"

covid_seq: .asciz
"AUUAAAGGUUUAUACCUUCCCAGGUAACAAACCAACCAACUUUCGAUCUCUUGUAGAUCUGUUCUCUAAACGAACUUUA ..."

.section .bss
input_buf: .space MAX_INP_LEN
.align 2

.section .text
main:
    bl Hardware_Init

    ldr r0, =msg_init
    bl Print_String
    ldr r0, =msg_prompt
    bl Print_String

    ldr r4, =input_buf
    mov r5, #0

Main_Loop:
    @ Hardware State Evaluation (Polling PD4)
    bl Check_Button_PD4
    cmp r0, #1
    beq Reset_Handler

    @ Serial Peripheral I/O Tracking (Non-blocking)
    bl UART2_ReceiveChar
    cmp r0, #0
    beq Main_Loop          @ Αν δεν ήρθε χαρακτήρας, συνέχισε το polling

    @ Check Enter (\n = 10, \r = 13)
    cmp r0, #10
    beq Process_Search
    cmp r0, #13
    beq Process_Search

    @ Filter for validation of chars in RNA
    cmp r0, #'A'
    beq Valid_Char
    cmp r0, #'G'
    beq Valid_Char
    cmp r0, #'U'
    beq Valid_Char
    cmp r0, #'C'
    beq Valid_Char
    b Main_Loop          @ ignore invalid chars

```

Valid_Char:

```
@ check for buffer overflow (MAX_INP_LEN - 1)
ldr r1, =MAX_INP_LEN
sub r1, r1, #1
cmp r5, r1
bge Main_Loop

@ Get complement
bl Get_Complement
strb r0, [r4, r5]
add r5, r5, #1          @ input_idx++

bl Get_Complement
bl UART2_SendChar

b Main_Loop
```

Reset_Handler:

Wait_Release:

```
bl Check_Button_PD4 @polling if pd4 is pressed
cmp r0, #1
beq Wait_Release

@ LED PA13 on & pd4 is not hold any more
ldr r0, =GPIOA_ODR
ldr r1, [r0]
orr r1, r1, #LED_PA13
str r1, [r0]

@ Delay 3000 ms = 3 sec
ldr r0, =3000
bl Delay_ms

@ Turn off LED PA13
ldr r0, =GPIOA_ODR
ldr r1, [r0]
bic r1, r1, #LED_PA13
str r1, [r0]

@ Clear Buffer - Reset the counting
mov r5, #0          @ input_idx = 0
bl Clear_Buffer

@ Print Abort Message
ldr r0, =msg_abort
bl Print_String

b Main_Loop
```

@ strstr & sprintf

Process_Search:

```
cmp r5, #0
beq Main_Loop          @ if buffer is cleaned, ignore enter

mov r0, #0
strb r0, [r4, r5]      @ Null-terminate input_buf

ldr r0, =msg_search
bl Print_String
mov r0, r4
bl Print_String
ldr r0, =msg_crlf
bl Print_String

mov r6, #0              @ r6 = Matches Counter
ldr r7, =covid_seq     @ r7 = covid_seq (ptr) same as main.c
```

Search_Loop:

```
mov r0, r7              @ current covid_seq
mov r1, r4
bl String_Search        @ custom strstr

cmp r0, #0              @ if returns 0, end search
beq End_Search

add r6, r6, #1          @ matches++

@ Position = (match_ptr - covid_seq) + 1
ldr r1, =covid_seq
sub r2, r0, r1          @ r2 = Offset
add r2, r2, #1          @ r2 = Position

@ Print Position
push {r0}               @ store match_ptr
ldr r0, =msg_match
bl Print_String
mov r0, r2
bl Print_Number          @ %d of sprintf
ldr r0, =msg_crlf
bl Print_String
pop {r0}                @ match_ptr

mov r7, r0
add r7, r7, #1          @ ptr++ , find the next one
b Search_Loop
```

End_Search:

```
ldr r0, =msg_crlf
bl Print_String
ldr r0, =msg_total
bl Print_String
```

```

mov r0, r6
bl Print_Number
ldr r0, =msg_crlf
bl Print_String
bl Print_String

```

```

@ Check PA12 for Matches > 0
cmp r6, #0
ble Reset_And_Prompt

```

```

@ Turn on PA12
ldr r0, =GPIOA_ODR
ldr r1, [r0]
orr r1, r1, #LED_PA12
str r1, [r0]

```

```

ldr r0, =200000
bl Delay_ms

```

```

@ Turn off PA12
ldr r0, =GPIOA_ODR
ldr r1, [r0]
bic r1, r1, #LED_PA12
str r1, [r0]

```

Reset_And_Prompt:

```

mov r5, #0
bl Clear_Buffer
ldr r0, =msg_prompt
bl Print_String
b Main_Loop

```

@ Subroutine: Get_Complement

Get_Complement:

```

cmp r0, #'A'
bne 1f
mov r0, #'U'
bx lr
1: cmp r0, #'U'
bne 2f
mov r0, #'A'
bx lr
2: cmp r0, #'G'
bne 3f
mov r0, #'C'
bx lr
3: cmp r0, #'C'
bne 4f
mov r0, #'G'
bx lr
4: mov r0, #0
bx lr

```

```

@ Subroutine: String_Search (strstr implementation)
@ Input: r0, r1
@ Output: r0 = Pointer to match, or 0 if not found

```

```
String_Search:
```

```

    push {r4-r7, lr}
    mov r4, r0          @ r4 = h_ptr

```

```
Outer_Search:
```

```

    ldrb r5, [r4]
    cmp r5, #0
    beq Not_Found      @ End of RNA, δεν βρέθηκε

```

```

    mov r5, r4          @ r5 = h_temp
    mov r6, r1          @ r6 = n_temp

```

```
Inner_Search:
```

```

    ldrb r7, [r6]
    cmp r7, #0
    beq Found_Match    @ match

```

```

    ldrb r2, [r5]
    cmp r2, r7
    bne Next_Outer     @ chars different, move onto the next byte

```

```

    add r5, r5, #1
    add r6, r6, #1
    b Inner_Search

```

```
Next_Outer:
```

```

    add r4, r4, #1      @ h_ptr++
    b Outer_Search

```

```
Found_Match:
```

```

    mov r0, r4          @ return counter
    pop {r4-r7, pc}

```

```
Not_Found:
```

```

    mov r0, #0
    pop {r4-r7, pc}

```

```
@ Subroutine: Print_Number
```

```
@ Input: r0 = Integer value
```

```
Print_Number:
```

```

    push {r4-r7, lr}
    cmp r0, #0
    bne 1f
    mov r0, #'0'
    bl UART2_SendChar
    pop {r4-r7, pc}

```

```

1: mov r4, r0          @ int
    mov r5, #0          @ digit counter

```

```

    mov r6, #10          @ divisor
Div_Loop:
    udiv r7, r4, r6      @ r7 = r4 / 10
    mls r1, r7, r6, r4   @ r1 = r4 - (r7 * 10) (mod)
    push {r1}           @ Push σ to stack
    add r5, r5, #1
    mov r4, r7
    cmp r4, #0
    bne Div_Loop

```

```

Print_Dig_Loop:
    pop {r0}
    add r0, r0, #'0'
    bl UART2_SendChar
    subs r5, r5, #1
    bne Print_Dig_Loop

```

```

    pop {r4-r7, pc}

```

```

@ Subroutine: Print_String
@ Input: r0 = Pointer to string

```

```

Print_String:
    push {r4, lr}
    mov r4, r0
PStr_Loop:
    ldrb r0, [r4]
    cmp r0, #0
    beq PStr_End
    bl UART2_SendChar
    add r4, r4, #1
    b PStr_Loop

```

```

PStr_End:
    pop {r4, pc}

```

```

@ Subroutine: Clear_Buffer

```

```

Clear_Buffer:
    push {r4, r5, lr}
    ldr r4, =input_buf
    mov r5, #MAX_INP_LEN
    mov r0, #0

```

```

Clr_Loop:
    strb r0, [r4]
    add r4, r4, #1
    subs r5, r5, #1
    bne Clr_Loop
    pop {r4, r5, pc}

```

```

@ Subroutine: Delay_ms

```

```

Delay_ms:
    push {r4, r5}

```

```

    ldr r4, =1600
    mul r4, r0, r4      @ r4 = ms * 1600
    mov r5, #0
D_Loop:
    cmp r5, r4
    bge D_End
    nop
    add r5, r5, #1
    b D_Loop
D_End:
    pop {r4, r5}
    bx lr

```

Ανάλυση main_asm.s:

1. Σκοπός και Αρχιτεκτονική Περιγραφή

Ο κώδικας του αρχείου main.s υλοποιεί την κεντρική λειτουργία του συστήματος βιοπληροφορικής, βασισμένος αποκλειστικά σε **ARM Assembly (Thumb-2)**. Με τον τρόπο αυτό, εξαλείφεται κάθε ανάγκη για τις βιβλιοθήκες της C (όπως string.h και stdio.h). Η εφαρμογή οργανώνεται γύρω από έναν αδιάλειπτο βρόχο ελέγχου (Main Loop), ο οποίος επιτηρεί τα εξωτερικά συμβάντα μέσω polling, διαχειρίζεται τη σειριακή θύρα και εκτελεί τους αλγορίθμους αναζήτησης απευθείας στους πόρους της μνήμης.

2. Διαχείριση Καταστάσεων και Δειγματοληψία (Polling)

Σε αντίθεση με τη χρήση εξωτερικών διακοπών, η ανάγνωση του διακόπτη επαναφοράς **PD4** πραγματοποιείται μέσω **non-blocking polling**.

- **Βρόχος Ελέγχου:** Κατά την έναρξη κάθε κύκλου στο Main_Loop, εκτελείται η ρουτίνα Check_Button_PD4. Σε περίπτωση ανίχνευσης λογικού '1', το πρόγραμμα μεταβαίνει άμεσα στη διαδικασία Reset_Handler.
- **Μηχανισμός Reset:** Ο Reset_Handler διασφαλίζει την απελευθέρωση του πλήκτρου μέσω της Wait_Release. Ακολούθως, ενεργοποιείται το LED στον **PA13**, εισάγεται καθυστέρηση 3 δευτερολέπτων, μηδενίζεται ο input_idx και εκκαθαρίζεται ο buffer για την επαναφορά του συστήματος.

3. Σειριακή Διεπαφή και Φιλτράρισμα RNA

Η λήψη των δεδομένων εκτελείται αδιάλειπτα μέσω της συνάρτησης UART2_ReceiveChar.

- **Φιλτράρισμα Δεδομένων:** Κάθε εισερχόμενος χαρακτήρας συγκρίνεται με τις έγκυρες βάσεις {A, G, U, C} και τους χαρακτήρες τερματισμού (\r, \n). Οποιοδήποτε άλλο στοιχείο απορρίπτεται άμεσα από τον ελεγκτή.
- **Αντιστοίχιση Βάσεων (Complement):** Μέσω της `Get_Complement`, το σύστημα υπολογίζει το συμπλήρωμα κάθε βάσης, το αποθηκεύει στον `input_buf` και το επιστρέφει ως `echo` στο τερματικό του χρήστη.

4. Προσαρμοσμένες Υπορουτίνες

Για την πλήρη αυτονομία από εξωτερικές βιβλιοθήκες, υλοποιήθηκαν εξειδικευμένες ρουτίνες σε χαμηλό επίπεδο:

- **Αλγόριθμος Αναζήτησης (String_Search):** Αποτελεί μια native υλοποίηση της `strstr`. Χρησιμοποιεί φωλιασμένους βρόχους σάρωσης για τη σύγκριση της ακολουθίας `covid_seq` με τον `buffer` εισόδου, επιστρέφοντας τη θέση της ταύτισης με ελάχιστη επιβάρυνση στη στοίβα.
- **Μορφοποίηση Ακεραίων (Print_Number):** Αντικαθιστά τη λειτουργία `sprintf`. Με τη χρήση των εντολών `UDIV` και `MLS`, υπολογίζει τα ψηφία του αριθμού, τα οποία ανακτώνται από τη στοίβα για την ορθή ASCII εκτύπωση των αποτελεσμάτων.
- **Καθυστέρηση Εξομοιωτή (Delay_ms):** Ένας βρόχος βασισμένος σε εντολές `nop`, ρυθμισμένος ειδικά για το περιβάλλον του Renode. Διασφαλίζει τον σωστό οπτικό συγχρονισμό των LED κατά τη λειτουργία του συστήματος.

Ο χρήστης πρέπει στο τερματικό να εκτελέσει την εξής εντολή:

```
sysbus.usart2 WriteLine "<to_be_matched_compl_string>"
```

Αυτό γίνεται καθώς στο `gui`, στο `UART2` δεν μπορεί να εισαχθεί σωστά ο χαρακτήρας αλλαγής γραμμής για να σηματοδοτήσει το τέλος εισόδου.

*Η **WriteLine** εισάγει χαρακτήρα αλλαγής γραμμής αυτόματα.*

Αποτελέσματα:

1. String's Complement to be matched: "U":

MICROLAB STM32-Renode GUI

connected, ODR 468.7 ms, IDR 9.6 ms

Full GPIOA/GPIOD. No MODER polling. ODR fast-poll, IDR slower-poll, optimistic button feedback.

GPIOA

IDR=0x00000000 ODR=0x00001000

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR	☐	☐	☐	☒	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
IDR	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
hold	hold	hold	hold	hold	hold	hold	hold	hold	hold	hold	hold	hold	hold	hold	hold	hold
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
pulse	p	p	p	p	p	p	p	p	p	p	p	p	p	p	p	p

GPIOD

IDR=0x00000000 ODR=0x00000000

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
IDR	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
hold	hold	hold	hold	hold	hold	hold	hold	hold	hold	hold	hold	hold	hold	hold	hold	hold
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
pulse	p	p	p	p	p	p	p	p	p	p	p	p	p	p	p	p

Renode Monitor Console

Paste commands here instead of using telnet. Multi-line commands are executed line at a time.

```
sysbus.usart2 WriteLine "U"
```

Send command(s) start pause machine Reset clear output

```
>>> sysbus.usart2 WriteLine "U"
>>> sysbus.usart2 WriteLine "U"
sysbus.usart2 WriteLine "U"
[33;1m(machine-0) [0m
```

UART2 Monitor

Click Attach UART2 after the platform is loaded.

Attach UART2

clear UART

```
Match found at position: 29891
Match found at position: 29892
Match found at position: 29893
Match found at position: 29894
Match found at position: 29895
Match found at position: 29896
Match found at position: 29897
Match found at position: 29898
Match found at position: 29899
Match found at position: 29900
Match found at position: 29901
Match found at position: 29902
Match found at position: 29903
Match found at position: 29904
Match found at position: 29905
Match found at position: 29906
Total matches: 8954
```

Enter RNA sequence:

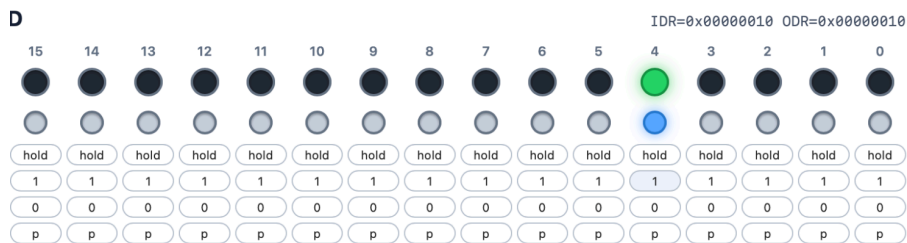
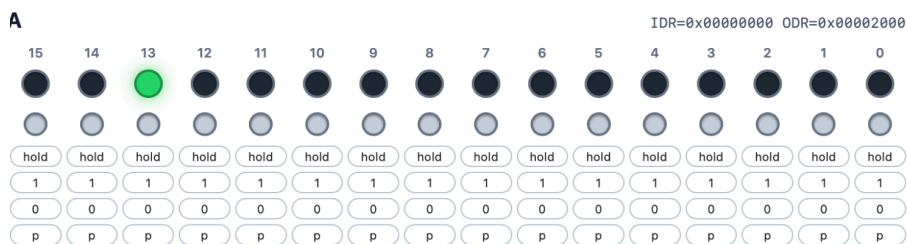
Text to send to virtual UART RX

Send

Παρατηρούμε ότι βρέθηκε το complenet του στη ακολουθία RNA του ιού και άναψε ορθώς το PA12 και βρέθηκαν 8954 matches, ακολουθόμενο το καθένα από τη θέση που βρέθηκε.

2. Για την ίδια ακολουθία (“U”) με συμπήρωμα “A” πατήσαμε το PD4, ανάβοντας ορθώς το λαμπάκι στο PA13 (εικόνα 1) και επανεκκινώντας την αναζήτηση (εικόνα 2).

JA/GPIOD. No MODER polling. ODR fast-poll, IDR slower-poll, optimistic button feedback.



Renode Monitor Console

Paste commands here instead of using tel line at a time.

```
sysbus.usart2 WriteLine "U"
```

Send command(s)

start

pause

```
>>> sysbus.usart2 WriteLine "U"
>>> sysbus.usart2 WriteLine "U"
sysbus.usart2 WriteLine "U"
[33;1m(machine-0) [0m
```

UART2 Monitor

Click Attach UART2 after the platform is loaded.

Attach UART2

clear UART

[RESET] Operation aborted. Enter new sequence:

Text to send to virtual UART RX

Send

3. Τέλος δοκιμάσαμε την ακολουθία (“UAGACA”) για να βρούμε το συμπλήρωμα (“AUCUGU”) της στο RNA του covid και παρατηρούμε ότι βρέθηκε σε διάφορες θέσεις (9), ανάβοντας ορθώς το λαμπάκι PA12 (εικόνα 1) και τυπώνοντας τα αποτελέσματα στο μέσω του UART2 (εικόνα 2).

The image shows the Renode Monitor Console interface. On the left, there is a row of 15 LEDs labeled 5 to 0. LED 12 is lit green. Below the LEDs are three rows of buttons: 'old', 'hold', and 'p'. The 'old' row has buttons labeled 1 to 14. The 'hold' row has buttons labeled 1 to 14. The 'p' row has buttons labeled 1 to 14. On the right, the 'Renode Monitor Console' window is open, showing the command 'sysbus.usart2 WriteLine "UAGACA"' and its output. The output shows the sequence 'UAGACA' being written to the UART2 port.

Renode Monitor Console

Paste commands here instead of using telnet. Multi-line at a time.

sysbus.usart2 WriteLine "UAGACA"

Send command(s) start pause machine

```
>>> sysbus.usart2 WriteLine "UAGACA"
sysbus.usart2 WriteLine "UAGACA"
[33;1m(machine-0) [0m
>>> sysbus.usart2 WriteLine "UAGACA"

>>> sysbus.usart2 WriteLine "UAGACA"
sysbus.usart2 WriteLine "UAGACA"
[33;1m(machine-0) [0m
>>> sysbus.usart2 WriteLine "UAGACA"

>>> sysbus.usart2 WriteLine "UAGACA"
```

```
Enter RNA sequence:
UAGACA
Searching for complement: AUCUGU
Match found at position: 60
Match found at position: 85
Match found at position: 1171
Match found at position: 8951
Match found at position: 15127
Match found at position: 18815
Match found at position: 20082
Match found at position: 20556
Match found at position: 22609
Total matches: 9
```

Θεωρητικές Ασκήσεις

ΑΣΚΗΣΗ 2

(α) Εκτίμηση Χωρητικότητας Flash Memory και SRAM

- **Flash Memory (Μη πτητική):** Εκεί αποθηκεύονται οι εντολές του επεξεργαστή και τα δεδομένα ανάγνωσης.
 - Το τμήμα **.text** (εντολές κώδικα) καταλαμβάνει **1 KB**.
 - Το τμήμα **.rodata** (σταθερές συμβολοσειρές και lookup tables) απαιτεί το πολύ **512 MB**.

Σημείωση: Αν και το section **.data** λειτουργεί στην SRAM, οι αρχικές του τιμές πρέπει να τηρούνται στη Flash για την αντιγραφή τους κατά την εκκίνηση. Συνεπώς, η Flash δεσμεύει επιπλέον το πολύ **512 MB**.

Ελάχιστη Flash: 1 KB (**.text**) + 512 MB (**.rodata**) + 512 MB (**.data init**) = **1024 MB + 1 KB**.

Χωρίς την υπόθεση για το initialization , η απαίτηση περιορίζεται σε 512 MB + 1 KB.

- **SRAM (Πτητική):** Χρησιμοποιείται για μεταβλητές που τροποποιούνται κατά το runtime.
 - Το τμήμα **.data** δεσμεύει **512 MB**.

(Η εξωτερική RAM διαχειρίζεται τα πακέτα των 128 MB, ενώ οι καταχωρητές GPIO αποτελούν memory-mapped I/O).

- **Ελάχιστη SRAM: 512 MB**

Εξαιρουμένων των αναγκών σε Stack και Heap.

Σημείωση: Αφού δεν έδινε η άσκηση κάποια πληροφορία για το **.bss**, υποθέσαμε πως αυτό πιάνει μηδενικό χώρο, δηλαδή δεν έχουμε uninitialized data.

(β) Memory Mapping και Χαρτογράφηση Διευθύνσεων

Σύμφωνα με τις προδιαγραφές του Cortex-M4, ο επεξεργαστής διαθέτει ενιαίο **fixed** χώρο διευθύνσεων 4 GB (0x00000000 - 0xFFFFFFFF). Η κατανομή ορίζεται ως εξής:

1. **.text & .rodata (Code Region):**

- 0x00000000 έως 0x1FFFFFFF.
- Θα χαρτογραφηθούν υποχρεωτικά το **.text** (1 KB) και το πρώτο τμήμα της Flash που περιέχει το **.rodata** (έως 512 MB), άμα χωράει. Σε αυτό το region (έως 512 MB) ο ARM περιμένει να βρει το Vector Table (στη διεύθυνση 0x00000000) και την Flash memory, διαβάζοντας τις εντολές απευθείας μέσω του I-Code bus για μέγιστη ταχύτητα.

2. **.data (SRAM Region):**

- 0x20000000 έως 0x3FFFFFFF.
- Χώρος για την εσωτερική SRAM, όπου πραγματοποιούνται οι λειτουργίες ανάγνωσης/γραφής δεδομένων μέσω του System Bus.

3. **GPIO Registers (Peripheral Region):**

- 0x40000000 έως 0x5FFFFFFF.
- Περιοχή υλοποίησης του Memory-Mapped I/O. Οι 80 bytes των GPIO καταλαμβάνουν θέσεις εδώ, με πρόσβαση μέσω εντολών LDR/STR.

4. **External Memory (Ext RAM Region):**

- 0x60000000 έως 0x9FFFFFFF.
- Καθορισμένο region για εξωτερικές μνήμες, απαραίτητο για τη διαχείριση εκτενών δεδομένων όπως τα πακέτα των 128 MB.

*Σημείωση: Αν λάβουμε υπόψιν ότι τα **.data** αρχικά πρέπει να υπάρξουν αρχικά στην flash -> Επειδή το Code Region εξαντλείται, το υπόλοιπο τμήμα της φυσικής Flash memory του SoC (που περιέχει τις **αρχικές τιμές του .data**, έως 512 MB ή **μέρος του .rodata**) θα χαρτογραφηθεί υποχρεωτικά στην αρχή του External Memory Space. Ο Cortex-M4 επιτρέπει κανονικά την ανάγνωση δεδομένων από εκεί κατά το boot.*

ΑΣΚΗΣΗ 3

Η εκφώνηση αναφέρει "1 κύκλο για ολοκλήρωση" για την εγγραφή (LED_CTRL), οπότε καταλαβαίνουμε ότι το περιφερειακό δεν απαιτεί επιπλέον κύκλους αναμονής (Wait States), άρα δέχεται τα δεδομένα αμέσως στην πρώτη φάση Access.

Αντίθετα, οι "2 κύκλοι για ολοκλήρωση" στην ανάγνωση (LED_STATUS) σημαίνουν ότι το περιφερειακό χρειάζεται μία επιπλέον καθυστέρηση 1 κύκλου ρολογιού (1 Wait State) για να φέρει τα δεδομένα, κρατώντας το σήμα PREADY σε λογικό '0'.

(α) Διαγράμματα Χρονισμού

Θα χρησιμοποιήσουμε τη πλατφόρμα **WaveDrom Editor** για κάθε μεταφορά δεδομένων από και προς το περιφερειακό μέσω του APB BUS για να τα παράξουμε και να τα οπτικοποιήσουμε.

Εγγραφή στο LED_CTRL (0x40000000) - Χωρίς Wait States:

Η εντολή STR r1, [r0] πυροδοτεί μια μεταφορά εγγραφής.

- Setup Phase (T1): Το PSEL γίνεται 1, το PWRITE γίνεται 1, και η διεύθυνση/δεδομένα είναι έγκυρα. Το PENABLE είναι 0.
- Access Phase (T2): Το PENABLE γίνεται 1. Επειδή η ολοκλήρωση θέλει 1 κύκλο, το περιφερειακό κρατάει το PREADY = 1. Η μεταφορά ολοκληρώνεται.

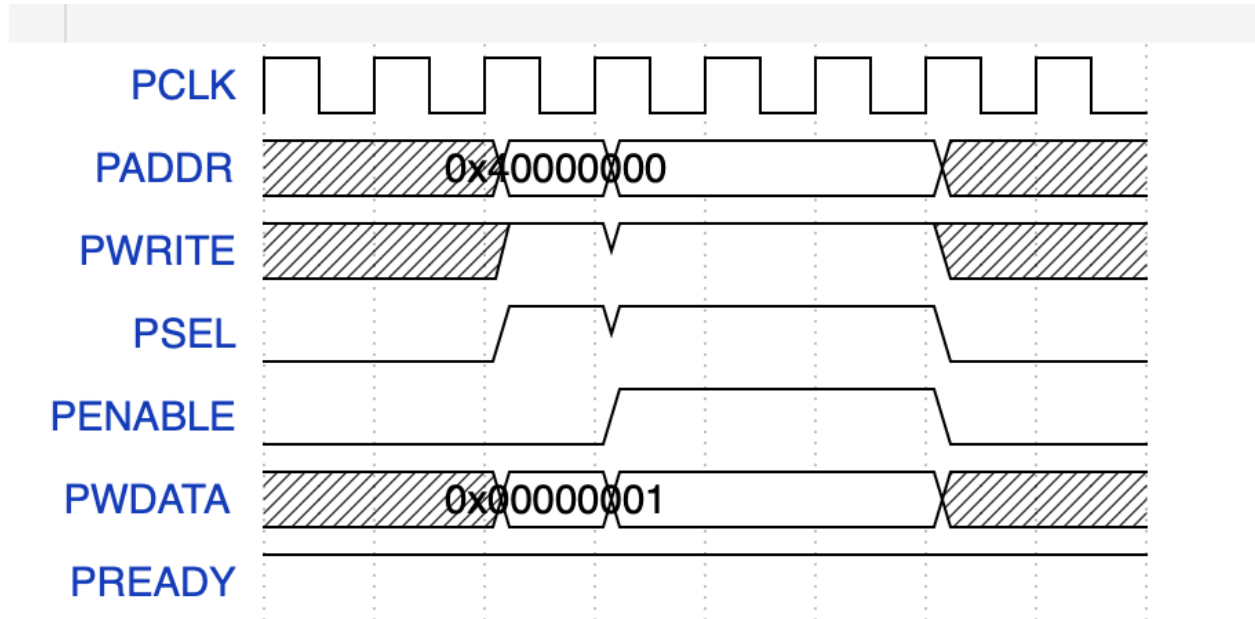
Θα χρησιμοποιήσουμε τη πλατφόρμα **WaveDrom Editor**.

WaveDrom Code:

```
{ signal: [  
  { name: 'PCLK', wave: 'p.....' },  
  { name: 'PADDR', wave: 'x.22..x.', data: ['0x40000000'] },  
  { name: 'PWRITE', wave: 'x.11..x.' },  
  { name: 'PSEL', wave: '0.11..0.' },  
  { name: 'PENABLE', wave: '0..1..0.' },  
  { name: 'PWDATA', wave: 'x.22..x.', data: ['0x00000001'] },  
  { name: 'PREADY', wave: '1.....' }
```

}}

Παρακάτω φαίνονται τα αποτελέσματα και το διάγραμμα χρονισμού για την εγγραφή στο LED_CTRL (0x40000000):



Ανάγνωση από το LED_STATUS (0x40000004) - Με 1 Wait State :

Η εντολή LDR r2, [r0] πυροδοτεί μεταφορά ανάγνωσης.

- Setup Phase (T1): Το PSEL γίνεται 1, το PWRITE γίνεται 0.
- Access Phase 1 - Wait (T2): Το PENABLE γίνεται 1. Το περιφερειακό χρειάζεται χρόνο, οπότε ρίχνει το PREADY στο 0. Ο διάυλος "παγώνει".
- Access Phase 2 - Ready (T3): Το περιφερειακό ανεβάζει το PREADY στο 1 και τοποθετεί την τιμή στο PRDATA

WaveDrom Code:

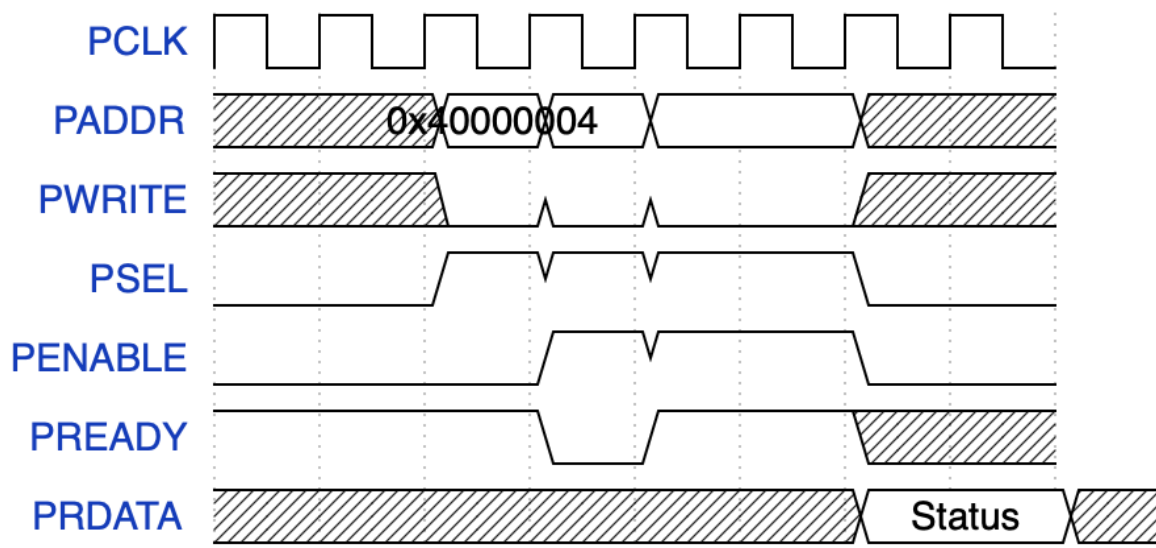
```
{ signal: [  
  { name: 'PCLK', wave: 'p.....' },  
  { name: 'PADDR', wave: 'x.222.x.', data: ['0x40000004'] },  
  { name: 'PWRITE', wave: 'x.000.x.' },  
  { name: 'PSEL', wave: '0.111.0.' },
```

```

{ name: 'PENABLE', wave: '0..11.0.' },
{ name: 'PREADY', wave: '1..01.x.' },
{ name: 'PRDATA', wave: 'x.....2.x', data: ['Status'] }
}

```

Παρακάτω φαίνονται τα αποτελέσματα και το διάγραμμα χρονισμού για την ανάγνωση από το LED_STATUS (0x40000004):



(β) Σχεδιασμός του FSM του Περιφερειακού APB

Βάσει των προδιαγραφών του AMBA APB, το περιφερειακό πρέπει να διαβάζει τα σήματα ελέγχου και να παράγει τα PREADY και PRDATA. Το FSM θα έχει τις εξής 4 καταστάσεις:

- **State 0: IDLE**
 - Το περιφερειακό βρίσκεται σε αδράνεια. Το PREADY μπορεί να είναι προεπιλεγμένα 1.
 - Μετάβαση: Όταν PSEL == 1 και PENABLE == 0, ο δίαυλος μπαίνει στη φάση SETUP. Το FSM εξετάζει τα PWRITE και PADDR.
 - Αν PADDR == 0x40000000 (Εγγραφή), πάμε στο State 1.
 - Αν PADDR == 0x40000004 (Ανάγνωση), πάμε στο State 2.
- **State 1: WRITE_ACCESS (0 Wait States)**
 - Εδώ ισχύει PENABLE == 1.
 - Ενέργεια: Διαβάζουμε το PWDATA και το γράφουμε στο LED hardware.

- Έξοδος: Θέτουμε $PREADY = 1$.
- Μετάβαση: Στον επόμενο κύκλο ρολογιού, επιστροφή στο IDLE.
- **State 2: READ_WAIT_CYCLE (1 Wait State)**
 - Εδώ ισχύει $PENABLE == 1$.
 - Ενέργεια: Το περιφερειακό διαβάζει την κατάσταση του LED (εσωτερική καθυστέρηση).
 - Έξοδος: Θέτουμε $PREADY = 0$ (για να παρατείνουμε το Access phase του διαύλου).
 - Μετάβαση: Στον επόμενο κύκλο ρολογιού, πάμε στο State 3.
- **State 3: READ_READY_CYCLE**
 - Εδώ ισχύει ακόμα $PENABLE == 1$ (λόγω του προηγούμενου wait state).
 - Έξοδος: Τοποθετούμε την τιμή του LED στο bus PRDATA. Θέτουμε $PREADY = 1$.
 - Μετάβαση: Η μεταφορά ολοκληρώθηκε. Στον επόμενο κύκλο, επιστροφή στο IDLE

Παρακάτω η οπτικοποίηση του FSM για τον ελεγκτή διεπαφής του APB περιφερειακού, σύμφωνα με τις προδιαγραφές του χρονισμού του APB:

